Table of Contents for React Notes

Chapter 1: Introduction to React

1. What is React?
2. Why Use React?
3. Features of React
4. Setting Up a React Project
5. Understanding JSX (JavaScript XML)
6. Virtual DOM vs Real DOM

Chapter 2: React Components & Props

1. Functional vs Class Components
2. Creating Functional Components
3. Creating Class Components
4. Understanding Props (Properties)
5. Passing Props Between Components
6. Default Props and PropTypes

Chapter 3: State and Lifecycle Methods

1. What is State?
2. State in Functional Components (useState Hook)
3. State in Class Components

4. Component Lifecycle Methods
    - Mounting (componentDidMount)
    - Updating (componentDidUpdate, shouldComponentUpdate)
    - Unmounting (componentWillUnmount)

Chapter 4: React Hooks (Modern React)

1. Introduction to Hooks
2. useState Hook (Managing State)
3. useEffect Hook (Handling Side Effects)
4. useRef Hook (Accessing DOM & Persisting Values)
5. useMemo Hook (Performance Optimization)
6. useCallback Hook (Optimizing Function References)
7. useReducer Hook (Managing Complex State)
8. Creating Custom Hooks

Chapter 5: React Context API & State Management

1. What is Context API?
2. Creating and Using Context
3. Context Provider and Consumer
4. Using useContext Hook
5. When to Use Context API vs Props
6. Introduction to Redux (Advanced)

Chapter 6: React Routing (Navigation)

1. What is React Router?

2. Setting Up React Router
3. Route, BrowserRouter, and Switch
4. Using Link & NavLink for Navigation
5. Nested Routes
6. Dynamic Routing with Parameters
7. Redirects and Protected Routes

Chapter 7: Handling Forms & User Input

1. Controlled vs Uncontrolled Components
2. Handling Form Inputs in React
3. Form Validation in React
4. Submitting Data using Fetch API
5. Using Actions for Form Submission

Chapter 8: Advanced React Concepts

1. Handling Events in React
2. Debouncing and Throttling in React
3. Higher Order Components (HOC)
4. Render Props Pattern
5. Error Boundaries in React
6. Lazy Loading & Code Splitting

Chapter 9: Working with APIs and Data Fetching

1. Fetching Data using Fetch API
2. Fetching Data using Axios
3. Using useEffect for Data Fetching

4. Handling API Errors
5. Caching API Data for Performance
6. Real-Time Data with WebSockets

Chapter 10: React Performance Optimization

1. Memoization with useMemo
2. Avoiding Unnecessary Re-renders
3. Using useCallback to Optimize Functions
4. React Profiler Tool
5. Code Splitting with React.lazy

Chapter 11: Authentication & Security in React

1. Handling Authentication in React
2. Using JWT for Authentication
3. Protecting Routes in React
4. Storing Tokens Securely
5. Role-Based Access Control (RBAC)

Chapter 12: Deploying React Applications

1. Building a Production-ready React App
2. Hosting React Apps on Netlify, Vercel, and GitHub Pages
3. Setting Up Continuous Deployment (CD)
4. Performance Optimization Before Deployment

📌 CHAPTER 1: INTRODUCTION TO REACT

1️⃣ React Kya Hai?

React ek **JavaScript library** hai jo **Facebook** ne banayi thi taake **fast, interactive, aur reusable user interfaces** ban sakein. Yeh **single-page applications (SPAs)** ke liye bohot zyada use hoti hai.
🛠️ **React ka main kaam hai UI banana aur efficiently update karna jab data change hota hai!**

🔹 **React sirf ek framework nahi hai!** Yeh ek **component-based library** hai jo sirf **View layer** handle karti hai, isi liye isko **frontend development** ke liye use kiya jata hai.

2️⃣ React Kyu Use Karein? 🤔

React itna **popular aur powerful** kyun hai? Yeh kuch reasons hain jo React ko **best frontend library** banate hain:

✅ **Component-Based Architecture** – App ko chhoti-chhoti cheezon mein tod kar manage karna asaan hota hai.
✅ **Fast Performance (Virtual DOM)** – Page reload nahi hota, sirf wo cheezein update hoti hain jo change hui hain.
✅ **Reusable Components** – Ek dafa koi component likh liya to use kahin bhi use kar sakte hain.
✅ **Large Community & Ecosystem** – Bohot saari **ready-made libraries** available hain jo React ke sath kaam karti hain.
✅ **Easy to Learn (Compared to Angular/Vue)** – Agar aapko JavaScript aati hai to React ko seekhna bohot easy hoga.

✅ **SEO Friendly** – React ki pages **server-side rendering** ke through search engines ke liye optimize ki ja sakti hain.

3️⃣ Features of React 🏆

🚀 **React ke kuch zabardast features:**

- 🔹 **JSX (JavaScript XML)** – JavaScript aur HTML ko mila kar likhne ka tareeqa.
- 🔹 **Virtual DOM** – Fast updates bina page reload kiye.
- 🔹 **One-Way Data Binding** – Data sirf ek taraf se flow karta hai, jisse debugging asaan hoti hai.
- 🔹 **Component-Based Architecture** – UI ko chhoti chhoti pieces mein tod kar manageable banana.
- 🔹 **React Hooks** – Functional components ko aur powerful banata hai (useState, useEffect etc.).
- 🔹 **React Router** – Page reload ke bina **navigation** ka system.
- 🔹 **React Redux & Context API** – **State management** ke liye powerful tools.

4️⃣ React Project Ka Setup Kaise Karein? ⚙️

React ka setup karne ke multiple tareeqe hain:

Method 1: Create React App (Recommended) 🏗️

Agar aap beginners hain to **Create React App (CRA)** best hai. Isko use karne ke liye aapke system mein **Node.js** installed hona chahiye.

Steps:

1️⃣ **Node.js Install Karein** – Download Node.js

2️⃣ **Terminal ya Command Prompt Open Karein**

3️⃣ **Run This Command:**

```
npx create-react-app my-app
cd my-app
npm start
```

🟢 Yeh command ek **ready-made React project** bana degi jo **localhost:3000** pe run karega.

Method 2: Manual Setup (Advanced Users) 🛠️

Agar aap **advanced setup** chahte hain to aap **React ko manually install** kar sakte hain:

```
mkdir my-react-app
cd my-react-app
npm init -y
npm install react react-dom
```

Ab **index.html** aur **index.js** ko manually setup karna padega.

5️⃣ JSX (JavaScript XML) Kya Hai?

JSX ka full form hai **JavaScript XML**, aur yeh ek tareeqa hai jisse aap **JavaScript ke andar HTML-like syntax likh sakte hain.**

🔹 **Bina JSX:**

```
const element = React.createElement('h1', null, 'Hello React!');
```

🔹 **JSX Ke Sath:**

```
const element = <h1>Hello React!</h1>;
```

✨ JSX ka use karne ka **main benefit** yeh hai ke code **zyada readable aur short ho jata hai.**

6️⃣ Virtual DOM vs Real DOM 🏎️💨

React **Virtual DOM (VDOM)** use karta hai jo Real DOM se **zyada fast hota hai.**

🔹 Feature	🏛️ Real DOM	⚡ Virtual DOM
Speed	Slow	Fast
Efficiency	Poor	High
Renders	Pura page reload hota hai	Sirf change hua part update hota hai
Performance	Heavy updates	Lightweight updates

🛠️ **Virtual DOM sirf wohi changes apply karta hai jo zaroori hote hain, isliye React bohot fast hota hai!**

🎯 Conclusion

🚀 **React ek powerful frontend library hai jo interactive aur high-performance web applications banane ke liye use hoti hai.** Iska **Virtual DOM**, **Component-Based Architecture**, aur **JSX** ise **fast, reusable, aur scalable** banata hai.

📌 CHAPTER 2: COMPONENTS & PROPS

1️⃣ Functional vs Class Components

React mein **do tareekay ke components** hote hain:

✅ Functional Components

Ek **simple function** jo JSX return karta hai. Yeh **lightweight** aur **fast** hote hain aur **React Hooks** ke sath state aur lifecycle handle kar sakte hain.

🔹 **Example:**

```
function MyComponent(props) {
  return <h1>Hello, {props.name}!</h1>;
}
```

🔹 **Key Features:**

- Simple aur **easy to write**
- **Hooks ke sath state aur lifecycle** handle kar sakte hain
- **Zyada optimize** aur **performance better**

✅ Class Components

Pehle React mein **state aur lifecycle** ko handle karne ke liye **Class Components** use hote the.

🔹 **Example:**

```
class MyComponent extends React.Component {
  render() {
```

```
    return <h1>Hello, {this.props.name}!</h1>;
  }
}
```

🔹 **Key Features:**

- **this.state aur lifecycle methods** ko support karte hain
- **Thoda complex** hote hain compared to functional components
- **Hooks aane ke baad kam use hote hain**

🔍 Comparison Table

Feature	Functional Component	Class Component
Syntax	Simple Function	Class aur `this` keyword
State	`useState` Hook ka use hota hai	`this.state` ka use hota hai
Lifecycle Methods	`useEffect` Hook ka use hota hai	`componentDidMount`, `componentDidUpdate` etc. ka use hota hai
Performance	Fast	Relatively Slow

Usage	Preferred in Modern React	Old Approach

2️⃣ Props (Properties) Kya Hain?

Props ka matlab hota hai **"properties"** jo ek component doosray component ko data pass karne ke liye use karta hai. Props **read-only** hote hain, yani component unko **modify nahi kar sakta**.

🔹 **Example:**

```
function MyComponent(props) {
  return <h1>Hello, {props.name}!</h1>;
}
```

🔹 **Props kaise pass karte hain?**

```
function App() {
  return <MyComponent name="Ali" />;
}
```

Jab `MyComponent` render hoga, to output hoga:

Hello, Ali!

🔹 **Props ke andar kya ho sakta hai?**

- **Strings:** `"Ali"`
- **Numbers:** `42`
- **Arrays:** `[1, 2, 3]`
- **Objects:** `{ name: "Ali" }`
- **Functions:** `() => console.log('Hello')`

🔹 **Props immutable hote hain**, yani ek component apne andar props ko **change nahi kar sakta**.

3️⃣ Default Props aur PropTypes

✅ Default Props

Agar kisi component ko **koi prop na mile**, to defaultProps ka use karke **default value** set kar sakte hain.

🔹 **Example:**

```
function Greeting(props) {
  return <h1>Hello, {props.name}!</h1>;
}

Greeting.defaultProps = {
  name: "Guest"
};
```

Agar `Greeting` component ko koi **name prop na mile**, to wo `"Guest"` show karega.

✅ PropTypes (Props ki Type Checking)

PropTypes ka use kar ke hum **ensure kar sakte hain ke props ka type sahi ho**.

🔹 **Example:**

```
import PropTypes from 'prop-types';


function Greeting(props) {
  return <h1>Hello, {props.name}!</h1>;
}


Greeting.propTypes = {
  name: PropTypes.string.isRequired
};
```

Agar `name` **string ke ilawa kuch aur hoga**, to console me warning aayegi.

🔹 **Different Types of PropTypes:**

PropType	Description
`PropTypes.string`	Sirf string allow hogi

`PropTypes.number`	Sirf number allow hoga
`PropTypes.bool`	Sirf true/false allow hoga
`PropTypes.array`	Sirf array allow hogi
`PropTypes.object`	Sirf object allow hoga
`PropTypes.func`	Sirf function allow hoga

🔹 **Default Props + PropTypes Combination Example:**

```
function Greeting(props) {
  return <h1>Hello, {props.name}!</h1>;
}

Greeting.defaultProps = {
  name: "Guest"
};
```

```
Greeting.propTypes = {
  name: PropTypes.string.isRequired
};
```

🎯 Conclusion

- **Functional Components** modern React me zyada use hote hain kyunki ye simple aur fast hote hain.
- **Props ek component ko doosray component se data pass karne ka tareeqa hai.**
- **Props ko change nahi kiya ja sakta, sirf use kiya jata hai.**
- **Agar koi prop missing ho, to defaultProps se uska default value set kar sakte hain.**
- **PropTypes se props ki type checking ki ja sakti hai.**

📌 CHAPTER 3: STATE AND LIFECYCLE

1️⃣State Kya Hoti Hai?

State ek **JavaScript object** hai jo component ke andar data store karta hai **jo time ke sath change ho sakta hai**. Jab state update hoti hai, to **React component automatically re-render hota hai**.

🔹 **Props aur State ka Difference:**

Feature	Props	State
Data Flow	Parent → Child	Component ke andar
Mutability	Immutable (Change nahi hoti)	Mutable (Change ho sakti hai)
Used in	Functional & Class Components	Functional (useState) & Class Components
Re-renders Component?	Nahi	Haan

2️⃣State in Functional Components (useState Hook)

Functional components me state manage karne ke liye `useState` hook ka use hota hai.

🔹 **Basic Example:**

```
import React, { useState } from 'react';


function Counter() {
  const [count, setCount] = useState(0);  // Initial state = 0


  function handleClick() {
    setCount(count + 1);  // State update karega
  }


  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={handleClick}>Increment</button>
    </div>
  );
}


export default Counter;
```

🔹 **Explanation:**

- `useState(0)`: Initial state set karega.
- `setCount(count + 1)`: Jab button click hoga, count **1 se increase** hoga.
- Jab bhi state change hogi, component **automatically re-render** hoga.

3️⃣ State in Class Components

Class components me state **object** ke form me define hoti hai aur `this.setState()` se update hoti hai.

🔹 **Example:**

```
class Counter extends React.Component {

  constructor(props) {

    super(props);

    this.state = { count: 0 };  // Initial state

  }



  handleClick = () => {

    this.setState({ count: this.state.count + 1 });  // State update karega

  }



  render() {

    return (
```

```
      <div>
        <p>Count: {this.state.count}</p>
        <button onClick={this.handleClick}>Increment</button>
      </div>
    );
  }
}
```

🔹 **Explanation:**

- `this.state = { count: 0 }`: Initial state set kar raha hai.
- `this.setState({ count: this.state.count + 1 })`: Jab button click hoga, count **1 se increase** hoga.

4️⃣ Component Lifecycle Methods

Lifecycle methods React ke **Class Components** me hoti hain, jo component ke **Mounting (start), Updating (change), aur Unmounting (destroy)** ke waqt chaltay hain.

✅ Mounting Phase (Jab Component Render Hota Hai)

🔹 `componentDidMount()`

- Jab component **pehli dafa render hota hai**, tab ye method call hota hai.
- Isme **API call** ya **DOM setup** kiya jata hai.

🔹 **Example:**

```
class MyComponent extends React.Component {
  componentDidMount() {
    console.log('Component mounted!');
  }

  render() {
    return <h1>Hello, World!</h1>;
  }
}
```

✅ Updating Phase (Jab Component Update Hota Hai)

🔹 `componentDidUpdate(prevProps, prevState)`

- Jab **state ya props update** hoti hain, tab ye method call hota hai.
- Isme **changes track** karne ke liye `prevProps` aur `prevState` ka use hota hai.

🔹 **Example:**

```
class MyComponent extends React.Component {
  componentDidUpdate(prevProps, prevState) {
    if (prevState.count !== this.state.count) {
      console.log('State updated!');
```

```
    }
  }
}
```

🔹 `shouldComponentUpdate(nextProps, nextState)`

- Agar aap **chahte hain ke unnecessary renders na ho**, to ye method `false` return kar sakta hai.

🔹 **Example:**

```
class MyComponent extends React.Component {
  shouldComponentUpdate(nextProps, nextState) {
    return nextState.count !== this.state.count;  // Sirf tab update hoga jab count change hoga
  }
}
```

✅ Unmounting Phase (Jab Component Remove Hota Hai)

🔹 `componentWillUnmount()`

- Jab component **DOM se remove hone wala ho**, tab ye method call hota hai.
- Isme **event listeners ya API calls ko cleanup** kiya jata hai.

🔹 **Example:**

```
class MyComponent extends React.Component {
  componentWillUnmount() {
    console.log('Component is being removed!');
  }
}
```

🎯 Conclusion

- **State ek mutable data hota hai jo component ke andar store hota hai.**
- **Functional components me `useState` hook se state manage hoti hai.**
- **Class components me `this.state` aur `this.setState()` ka use hota hai.**
- **Lifecycle methods Class Components me state aur events ko manage karne ke liye use hote hain.**

📌 CHAPTER 4: REACT HOOKS (MODERN REACT)

1️⃣ Introduction to Hooks 🪝

Hooks **React 16.8** me introduce hue jo functional components ko **state aur lifecycle features** use karne ki ijazat dete hain. Hooks ka **use class components ki complexity ko kam karna aur code ko clean banana hai**.

🔹 **Hooks se pehle:** Sirf **class components** state aur lifecycle methods use kar sakte the.

🔹 **Hooks ke baad:** Functional components bhi **state aur lifecycle** handle kar sakte hain.

🔹 **Rules of Hooks:**

✅ Hooks sirf **functional components ke andar** use hote hain.

✅ Hooks ko **loops, conditions ya nested functions ke andar nahi likhna chahiye**.

2️⃣ useState Hook (Managing State) 🔄

`useState` React me **state manage** karne ke liye use hota hai.

🔹 **Basic Syntax:**

```
const [state, setState] = useState(initialValue);
```

🔹 **Example:**

```
import React, { useState } from 'react';
```

```
function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
    </div>
  );
}

export default Counter;
```

🔹 **Key Points:**

- `count` = Current state
- `setCount` = Function jo **state update** karega
- `useState(0)` = Initial value **0** hai

3️⃣ useEffect Hook (Handling Side Effects) 🎭

`useEffect` React me **side effects** handle karne ke liye use hota hai jaise:
✅ API Calls

✅ DOM Manipulation
✅ Event Listeners

🔹 **Basic Syntax:**

```
useEffect(() => {

  // Side effect code

}, [dependencies]);
```

🔹 **Example (Data Fetching):**

```
import React, { useState, useEffect } from 'react';


function DataFetcher() {

  const [data, setData] = useState([]);


  useEffect(() => {

    fetch('https://jsonplaceholder.typicode.com/posts')

      .then(response => response.json())

      .then(data => setData(data));

  }, []);


  return (

    <div>
```

```
        <h1>Data List</h1>
        <ul>
          {data.map(item => (
            <li key={item.id}>{item.title}</li>
          ))}
        </ul>
      </div>
    );
}
```

🔹 **Key Points:**

- `useEffect(() => {...}, [])` – **Sirf ek baar run hoga** (component mount hone par).
- **Agar dependency array me koi value ho**, to wo value change hone par effect dobara chalega.

4️⃣ useRef Hook (Accessing DOM & Persisting Values) 📌

`useRef` ek mutable reference banata hai jo **DOM elements ya component ke re-renders ke beech persist** karta hai.

🔹 **Example (Input Field Focus):**

```
import React, { useRef } from 'react';
```

```
function FocusInput() {
  const inputRef = useRef(null);

  const handleFocus = () => {
    inputRef.current.focus();
  };

  return (
    <div>
      <input ref={inputRef} type="text" />
      <button onClick={handleFocus}>Focus Input</button>
    </div>
  );
}

export default FocusInput;
```

🔹 **Key Points:**

- `useRef(null)` ek **reference object return** karta hai
- `inputRef.current.focus()` directly **DOM element access** karta hai
- `useRef` state store karta hai **bina re-renders trigger kiye**

5️⃣ useMemo Hook (Performance Optimization) 🚀

`useMemo` **expensive calculations** ko optimize karne ke liye use hota hai.

🔹 **Example (Factorial Calculation):**

```
import React, { useState, useMemo } from 'react';


function ExpensiveComputation() {
  const [number, setNumber] = useState(0);


  const factorial = useMemo(() => {
    console.log('Calculating factorial...');
    let result = 1;
    for (let i = 1; i <= number; i++) {
      result *= i;
    }
    return result;
  }, [number]);


  return (
    <div>
```

```
        <p>Number: {number}</p>

        <p>Factorial: {factorial}</p>

        <button onClick={() => setNumber(number + 1)}>Increase</button>

      </div>

  );

}


export default ExpensiveComputation;
```

🔹 **Key Points:**

- `useMemo(() => {}, [dependencies])` sirf **jab dependency change ho tab function execute karega**
- **Recomputes only when necessary** (performance optimization)

6️⃣ useCallback Hook (Optimizing Function References) 🎯

`useCallback` ek function **memoize** karta hai taake wo **unnecessary re-renders na kare**.

🔹 **Example:**

```
import React, { useState, useCallback } from 'react';


function ParentComponent() {
```

```
  const [count, setCount] = useState(0);

  const increment = useCallback(() => {
    setCount(prev => prev + 1);
  }, []);

  return (
    <div>
      <p>Count: {count}</p>
      <ChildComponent increment={increment} />
    </div>
  );
}

function ChildComponent({ increment }) {
  console.log('Child component re-rendering');
  return <button onClick={increment}>Increment</button>;
}

export default ParentComponent;
```

🔹 **Key Points:**

- `useCallback(() => {}, [dependencies])` function ko **memoize karta hai**
- **Performance improve karta hai** by preventing unnecessary function re-creations

---

7️⃣ useReducer Hook (Managing Complex State) 🏗️

`useReducer` complex state logic ko handle karne ke liye `useState` ka alternative hai.

🔹 **Basic Syntax:**

```
const [state, dispatch] = useReducer(reducer, initialState);
```

🔹 **Example (Counter with useReducer):**

```
import React, { useReducer } from 'react';

const initialState = { count: 0 };

const reducer = (state, action) => {
  switch (action.type) {
    case 'increment':
      return { count: state.count + 1 };
    case 'decrement':
      return { count: state.count - 1 };
```

```
    default:
      return state;
  }
};

function Counter() {
  const [state, dispatch] = useReducer(reducer, initialState);

  return (
    <div>
      <p>Count: {state.count}</p>
      <button onClick={() => dispatch({ type: 'increment' })}>Increment</button>
      <button onClick={() => dispatch({ type: 'decrement' })}>Decrement</button>
    </div>
  );
}

export default Counter;
```

🔹 **Key Points:**

- `useReducer(reducer, initialState)` complex state ko handle karta hai

- `dispatch({ type: 'action' })` function ko execute karta hai

8️⃣ Creating Custom Hooks 🛠️

Agar koi logic **multiple components** me repeat ho raha ho, to usko ek **Custom Hook** me likh sakte hain.

🔹 **Example:**

```
import { useState, useEffect } from 'react';


function useFetch(url) {
  const [data, setData] = useState(null);

  useEffect(() => {
    fetch(url)
      .then(res => res.json())
      .then(setData);
  }, [url]);

  return data;
}

export default useFetch;
```

🔹 **Key Points:**

- Custom hooks **"use" se start** hote hain
- **Reusable logic** ko manage karne ke liye best hain

🎯 Conclusion

- **Hooks functional components ko state aur lifecycle manage karne ki power dete hain.**
- **useState aur useEffect sabse zyada commonly used hooks hain.**
- **useMemo, useCallback, aur useReducer performance aur complex state management ke liye use hote hain.**
- **Custom hooks se reusable logic likhna easy hota hai.**

📌 CHAPTER 5: REACT CONTEXT API & STATE MANAGEMENT 🏗️

1️⃣What is Context API? 🌍

React me **data ko pass karne ka traditional tareeqa props hai**, lekin **deeply nested components me props drilling ka issue** hota hai. **Context API** is problem ko solve karta hai **global state management ka ek simple aur efficient tareeqa** dekar.

✅ **Global state ko manage karta hai**
✅ **Multiple components ke beech data share karne me madad karta hai**
✅ **Props drilling ki zaroorat nahi hoti**

2️⃣Creating and Using Context 🏗️

React me Context create karne ke liye `createContext()` ka use hota hai.

🔹 **Step 1: Context Create Karna**

```
import React, { createContext } from "react";

// Context create kar rahe hain
const MyContext = createContext();

export default MyContext;
```

✅ **Ek global state ka instance create ho gaya**

3️⃣ Context Provider and Consumer 🏠

✅ **Provider ka kaam hai values ko provide karna**
✅ **Consumer ka kaam hai values ko consume karna**

🔹 **Step 2: Provider Setup Karna**

```
import React, { useState } from "react";
import MyContext from "./MyContext";

function MyProvider({ children }) {
  const [user, setUser] = useState("Ali");

  return (
    <MyContext.Provider value={{ user, setUser }}>
      {children}
    </MyContext.Provider>
  );
}

export default MyProvider;
```

✅ **`MyProvider` ek wrapper hai jo children components ko `user` value provide karega**

🔹 **Step 3: Consumer Component (Class-Based)**

```
import React from "react";
import MyContext from "./MyContext";

class MyComponent extends React.Component {
  render() {
    return (
```

```
        <MyContext.Consumer>
          {(context) => <h1>User: {context.user}</h1>}
        </MyContext.Consumer>
      );
    }
}

export default MyComponent;
```

✅ **Consumer `user` state ko access kar raha hai jo `Provider` ne diya**

4️⃣ Using `useContext` Hook 🎯

⚡ **Class components me Consumer ka use hota tha, lekin hooks ke aane ke baad `useContext` hook ziada popular hai.**

🔹 **Example (Functional Component using `useContext`)**

```
import React, { useContext } from "react";
import MyContext from "./MyContext";

function MyComponent() {
  const { user } = useContext(MyContext);

  return <h1>User: {user}</h1>;
}

export default MyComponent;
```

✅ **Zyada clean aur readable code hai**
✅ **Directly `useContext` ka use kar ke `user` value access ho rahi hai**

5️⃣ When to Use Context API vs Props? 🤔

Scenario	Use Context API 🌍	Use Props 🔗
Simple Parent-Child Data Transfer	❌	✅
Deeply Nested Components	✅	❌
Multiple Components ko Same Data Chahiye	✅	❌
Local State (Component Specific)	❌	✅
Global State Management	✅	❌

✅ **Agar data sirf ek component ke andar use ho raha hai to `props` sahi hai**
✅ **Agar data multiple components me chahiye to `Context API` best option hai**

6️⃣ Introduction to Redux (Advanced) ⚡

✅ **Redux ek powerful state management tool hai jo large-scale applications ke liye best hai**
✅ **Complex state management aur API calls ke saath efficiently kaam karta hai**

🔹 **Redux vs Context API**

Feature	Redux ⚡	Context API 🌍
Complexity	High	Low
Setup	Complicated	Simple
Best For	Large Apps	Medium/Small Apps

State Management	Centralized Store	Component-based State

✅ **Agar small/medium app hai to Context API kaafi hai**
✅ **Agar complex state management required hai to Redux better choice hai**

🎯 Conclusion

- **Context API global state ko manage karne ka best tareeqa hai**
- `createContext()` **se context create hota hai**
- `Provider` **values provide karta hai,** `Consumer` **values access karta hai**
- **Functional components me** `useContext` **ziada easy aur readable hai**
- **Redux large-scale applications ke liye Context API ka alternative hai**

🚀 **Agar project chhota hai to Context API use karein, lekin agar complex state management chahiye to Redux sahi rahega!**

📌 CHAPTER 6: REACT ROUTING (NAVIGATION) 🗺️

1️⃣What is React Router? 🔄

React Router ek powerful library hai jo React applications me client-side navigation ko handle karne ke liye use hoti hai.

✅ **Single Page Applications (SPA) me multiple pages ka illusion create karta hai**
✅ **Page refresh ke bina views switch karne ki facility deta hai**
✅ **Dynamic URLs aur nested routes ko support karta hai**

2️⃣Setting Up React Router ⚙️

React Router ko apne project me install karne ke liye npm ya yarn ka use karein:

- **npm install react-router-dom**

Ya phir yarn use karein:

- **yarn add react-router-dom**

✅ **Ye command `react-router-dom` package ko install karegi**
✅ **Ye package React applications me routing support karta hai**

3️⃣ Route, BrowserRouter, and Switch 🔀

✅ `BrowserRouter` → Routing setup ke liye wrapper component
✅ `Route` → Specific URL ke liye component render karta hai
✅ `Switch` (Deprecated, use `Routes`) → Ek waqt me sirf ek matching route render karta hai

🔹 Example (Basic Routing with `Routes`)

```
import React from "react";
import { BrowserRouter, Routes, Route } from "react-router-dom";
import Home from "./Home";
import About from "./About";
import Contact from "./Contact";

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/contact" element={<Contact />} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

✅ `path="/"` Home component ko render karega
✅ `Routes` ek waqt me sirf ek route ko render karega

4️⃣ Using `Link` & `NavLink` for Navigation 🔗

✅ `Link` → Traditional `<a>` tags ka React version, jo page reload nahi hone deta
✅ `NavLink` → Active route ke liye CSS class apply karta hai

🔹 Example (`Link` Component for Navigation)

```
import React from "react";
import { Link } from "react-router-dom";

function Navbar() {
  return (
    <nav>
      <ul>
        <li><Link to="/">Home</Link></li>
        <li><Link to="/about">About</Link></li>
        <li><Link to="/contact">Contact</Link></li>
      </ul>
    </nav>
  );
}

export default Navbar;
```

✅ Page refresh ke bina URL change karega
✅ SPA ka smooth experience milega

🔹 Example (`NavLink` for Active Styling)

```
import React from "react";
import { NavLink } from "react-router-dom";

```

```
function Navbar() {
  return (
    <nav>
      <ul>
        <li>
          <NavLink to="/" className={({ isActive }) => (isActive ? "active" : "")}>
            Home
          </NavLink>
        </li>
        <li>
          <NavLink to="/about" className={({ isActive }) => (isActive ? "active" : "")}>
            About
          </NavLink>
        </li>
        <li>
          <NavLink to="/contact" className={({ isActive }) => (isActive ? "active" : "")}>
            Contact
          </NavLink>
        </li>
      </ul>
    </nav>
  );
}

export default Navbar;
```

✅ Active route ke liye class apply karega

✅ Dynamic styling possible hai

5️⃣ Nested Routes 🏗️

✅ Ek parent route ke andar multiple child routes define kiye ja sakte hain
✅ Dynamic layouts aur dashboard-style navigation ke liye useful hai

🔹 Example (Nested Routes Implementation)

```
import React from "react";
import { BrowserRouter, Routes, Route, Outlet } from "react-router-dom";

function Dashboard() {
  return (
    <div>
      <h2>Dashboard</h2>
      <Outlet /> {/* Ye nested routes ka content render karega */}
    </div>
  );
}

function Profile() {
  return <h3>My Profile</h3>;
}

function Settings() {
  return <h3>Settings</h3>;
}

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/dashboard" element={<Dashboard />}>
          <Route path="profile" element={<Profile />} />
          <Route path="settings" element={<Settings />} />
```

```
            </Route>
          </Routes>
        </BrowserRouter>
      );
    }

    export default App;
```

✅ `Outlet` nested components ko render karta hai
✅ `/dashboard/profile` ya `/dashboard/settings` navigate karne pe sahi component render hoga

6️⃣ Dynamic Routing with Parameters 🏷️

✅ React Router me URLs ke andar parameters pass kiye ja sakte hain
✅ Ye feature dynamically generated pages ke liye useful hai (e.g., product pages, user profiles)

🔹 Example (Dynamic Routes with Parameters)

```
import React from "react";
import { useParams } from "react-router-dom";

function UserProfile() {
  let { id } = useParams();
  return <h2>User ID: {id}</h2>;
}

export default UserProfile;
```

🔹 **Routes Setup for Dynamic Routing**

```
<Routes>
  <Route path="/user/:id" element={<UserProfile />} />
</Routes>
```

✅ Agar user `/user/123` visit karega, to `User ID: 123` show hoga
✅ `useParams()` se URL me se dynamic parameter fetch ho raha hai

7️⃣ Redirects and Protected Routes 🔐

✅ Kuch routes sirf authenticated users ke liye accessible hone chahiye
✅ Unauthorized users ko `/login` par redirect kiya jaye

🔹 **Example (Protected Route - Redirect If Not Logged In)**

```
import React from "react";
import { Navigate } from "react-router-dom";

function ProtectedRoute({ user, children }) {
  return user ? children : <Navigate to="/login" />;
}

export default ProtectedRoute;
```

🔹 **Using ProtectedRoute Component**

```
<Routes>
  <Route path="/dashboard" element={<ProtectedRoute user={isLoggedIn}><Dashboard /></ProtectedRoute>} />
</Routes>
```

✅ Agar `isLoggedIn === false` hoga, to user ko `/login` par bhej diya jayega
✅ Agar `isLoggedIn === true` hoga, to `Dashboard` component show hoga

🎯 Conclusion

- React Router client-side routing ko enable karta hai
- BrowserRouter ko wrap karna zaroori hai taake routing work kare
- `Link` & `NavLink` traditional anchor tags ka better alternative hain
- Nested routes se better component structuring hoti hai
- Dynamic routing URLs ko flexible banata hai
- Protected routes se authentication manage hoti hai

🚀 Routing ke bina React apps incomplete lagti hain! Ab aap navigation ko efficiently implement kar sakte hain.

📌 CHAPTER 7: HANDLING FORMS & USER INPUT ✍️

1️⃣ Controlled vs Uncontrolled Components 🎛️

React me forms do tareeke se handle kiye ja sakte hain:
✅ Controlled Components → State ke zariye input control hota hai
✅ Uncontrolled Components → Direct DOM ke zariye input control hota hai

Feature	Controlled Components 👨‍💻	Uncontrolled Components 🔄
Data Handling	React `useState` ya `useReducer` se handle hota hai	Directly DOM ke `ref` se handle hota hai
Re-rendering	Har input change par re-render hota hai	Re-render nahi hota, direct DOM me change hota hai
Recommended For	Most use cases, better control	Large forms jahan React state overhead ho sakti hai

🔹 Example (Controlled Component - State Based)

```
import React, { useState } from "react";
```

```
function ControlledForm() {
  const [name, setName] = useState("");


  return (
    <div>
      <input
        type="text"
        value={name}
        onChange={(e) => setName(e.target.value)}
        placeholder="Enter Name"
      />
      <p>Name: {name}</p>
    </div>
  );
}


export default ControlledForm;
```

✅ State ke through input ka data React handle kar raha hai
✅ Har change par onChange function se update ho raha hai

🔹 **Example (Uncontrolled Component - Using Ref)**

```
import React, { useRef } from "react";


function UncontrolledForm() {
  const inputRef = useRef(null);


  function handleSubmit() {
    alert("Entered Name: " + inputRef.current.value);
  }


  return (
    <div>
      <input type="text" ref={inputRef} placeholder="Enter Name" />
      <button onClick={handleSubmit}>Submit</button>
    </div>
  );
}


export default UncontrolledForm;
```

✅ Directly `ref` se input value fetch ho rahi hai, bina state update kiye
✅ React state maintain nahi karta, sirf DOM se value uthata hai

---

2️⃣ Handling Form Inputs in React 📝

Jab forms handle karte hain, to multiple inputs ko manage karna zaroori hota hai.

🔹 Example (Multiple Input Fields in Form - Controlled)

```
import React, { useState } from "react";


function MultiFieldForm() {
  const [formData, setFormData] = useState({ name: "", email: "" });


  function handleChange(e) {
    setFormData({ ...formData, [e.target.name]: e.target.value });
  }


  return (
    <form>
      <input
        type="text"
        name="name"
        value={formData.name}
```

```
        onChange={handleChange}
        placeholder="Enter Name"
      />
      <input
        type="email"
        name="email"
        value={formData.email}
        onChange={handleChange}
        placeholder="Enter Email"
      />
      <p>Name: {formData.name}</p>
      <p>Email: {formData.email}</p>
    </form>
  );
}

export default MultiFieldForm;
```

✅ Multiple fields ko ek `formData` object me store kiya gaya hai
✅ `onChange` handler dynamically data update kar raha hai

3️⃣Form Validation in React ✅❌

✅ Form validation kaafi zaroori hoti hai taake user valid data enter kare
✅ React me validation client-side aur server-side dono levels pe hoti hai

🔹 Example (Basic Form Validation - Required Fields & Email Format)

```
import React, { useState } from "react";


function FormValidation() {

  const [email, setEmail] = useState("");

  const [error, setError] = useState("");


  function validateEmail(e) {

    const value = e.target.value;

    setEmail(value);


    if (!value.includes("@")) {

      setError("Invalid Email!");

    } else {

      setError("");

    }

  }
```

```
  function handleSubmit(e) {
    e.preventDefault();
    if (error) {
      alert("Fix errors before submitting!");
    } else {
      alert("Form Submitted Successfully!");
    }
  }

  return (
    <form onSubmit={handleSubmit}>
      <input type="email" value={email} onChange={validateEmail}
placeholder="Enter Email" />
      {error && <p style={{ color: "red" }}>{error}</p>}
      <button type="submit">Submit</button>
    </form>
  );
}

export default FormValidation;
```

✅ Email field me @ hona zaroori hai warna error message show hoga
✅ `handleSubmit` me validation ka final check lagaya gaya hai

4️⃣ Submitting Data using Fetch API 📡

Jab form submit hota hai, to uska data backend server ya API ko bhejna hota hai. Fetch API se ye request bhej sakte hain.

🔹 Example (Submitting Form Data using Fetch API - POST Request)

```
import React, { useState } from "react";


function SubmitForm() {
  const [name, setName] = useState("");


  function handleSubmit(e) {
    e.preventDefault();


    fetch("https://example.com/api/submit", {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify({ name })
```

```
    })
    .then(response => response.json())
    .then(data => console.log("Success:", data))
    .catch(error => console.error("Error:", error));
  }

  return (
    <form onSubmit={handleSubmit}>
      <input type="text" value={name} onChange={(e) =>
setName(e.target.value)} placeholder="Enter Name" />
      <button type="submit">Submit</button>
    </form>
  );
}

export default SubmitForm;
```

✅ Data JSON format me backend ko bheja ja raha hai
✅ Error handling implemented hai taake errors console me log ho sakein

---

5️⃣ Using Actions for Form Submission 🚀

Agar hum React Router ka use karein, to actions ko form submission handle karne ke liye use kiya ja sakta hai.

🔹 Example (React Router Actions for Form Handling)

```
import { Form, useActionData } from "react-router-dom";


export async function submitAction({ request }) {
  const formData = await request.formData();
  const name = formData.get("name");


  return { success: `Form submitted successfully for ${name}!` };
}


function ActionForm() {
  const data = useActionData();


  return (
    <div>
      <Form method="post">
        <input type="text" name="name" placeholder="Enter Name" />
        <button type="submit">Submit</button>
      </Form>
```

```
      {data && <p>{data.success}</p>}
    </div>
  );
}

export default ActionForm;
```

✅ Form submission directly backend API pe process ho sakti hai
✅ React Router actions async data fetching aur submission ko simplify karte hain

---

🎯 Conclusion

- Controlled components React state ka use karke input handle karte hain
- Uncontrolled components direct DOM se input handle karte hain
- Form validation user input ko check karne aur errors handle karne ke liye zaroori hoti hai
- Fetch API se form data backend ya database me bhejna possible hota hai
- React Router actions se form submission ko streamline kiya ja sakta hai

🚀 Efficient form handling se user experience aur performance improve hoti hai!

📌 CHAPTER 8: ADVANCED REACT CONCEPTS 🚀

1️⃣ Handling Events in React 🎯

React me event handling **JavaScript se thoda different hota hai**. **React synthetic events ka use karta hai** jo **cross-browser compatibility provide karte hain**.

✅ **Events camelCase me likhe jate hain** (e.g., `onClick`, `onChange`)
✅ **Event handler functions directly pass hote hain**
✅ **Event object automatically pass hota hai**

🔹 **Example (Handling Click Event)**

```
function MyButton() {
  function handleClick() {
    alert("Button clicked!");
  }

  return <button onClick={handleClick}>Click Me</button>;
}
```

🔹 **Passing Parameters in Event Handlers**

```
function MyButton() {
  function handleClick(name) {
    alert("Hello, " + name);
  }

  return <button onClick={() => handleClick("Ali")}>Click Me</button>;
}
```

✅ **Arrow function se extra arguments pass karna possible hai**

🔹 **Preventing Default Behavior**

```
function MyForm() {
  function handleSubmit(event) {
    event.preventDefault(); // Page reload ko rok raha hai
    alert("Form submitted!");
  }

  return <form onSubmit={handleSubmit}>
    <button type="submit">Submit</button>
  </form>;
}
```

✅ `event.preventDefault()` **default event behavior ko rokta hai**

2️⃣ Debouncing & Throttling in React 🕑

🔵 Debouncing (Delay Execution Until User Stops Typing)

✅ **Useful for Search Input Fields (API Calls Optimize Karne ke liye)**
✅ **Multiple function calls ko ek optimized call me convert karta hai**

🔹 **Example (Debouncing Search Input with `useEffect`)**

```
import React, { useState, useEffect } from "react";

function SearchBox() {
  const [query, setQuery] = useState("");
  const [debouncedQuery, setDebouncedQuery] = useState("");
```

```
  useEffect(() => {
    const timer = setTimeout(() => {
      setDebouncedQuery(query);
    }, 500); // 500ms ka delay

    return () => clearTimeout(timer);
  }, [query]);

  return (
    <input
      type="text"
      value={query}
      onChange={(e) => setQuery(e.target.value)}
      placeholder="Type to search..."
    />
  );
}

export default SearchBox;
```

✅ **500ms ke delay ke baad API call hogi**
✅ **Performance improve hoti hai**

🔵 Throttling (Limit Function Execution)

✅ **Useful for Scroll Events (e.g., infinite scrolling)**
✅ **Function execution ko ek specific interval tak restrict karta hai**

🔹 **Example (Throttling Scroll Event)**

```
import React, { useEffect } from "react";
```

```
function ThrottledScroll() {
  useEffect(() => {
    let lastCall = 0;
    const handleScroll = () => {
      const now = new Date().getTime();
      if (now - lastCall > 1000) { // 1 second throttle
        console.log("User Scrolled!");
        lastCall = now;
      }
    };

    window.addEventListener("scroll", handleScroll);
    return () => window.removeEventListener("scroll", handleScroll);
  }, []);

  return <h2>Scroll the Page</h2>;
}

export default ThrottledScroll;
```

✅ **1-second ke interval me hi event trigger hoga**
✅ **Performance improve hogi by reducing event calls**

3️⃣ Higher Order Components (HOC) 🎭

HOC ek **function hai jo ek component ko input leta hai aur naye enhanced component return karta hai**.

✅ **Code reuse kaafi easy ho jata hai**
✅ **Common logic ko multiple components ke beech share karna easy hota hai**

🔹 **Example (HOC for Authentication Check)**

```
function withAuth(Component) {
  return function AuthenticatedComponent(props) {
    const isAuthenticated = true; // Authentication Logic
    return isAuthenticated ? <Component {...props} /> : <h2>Please Login</h2>;
  };
}

function Dashboard() {
  return <h2>Welcome to Dashboard!</h2>;
}

export default withAuth(Dashboard);
```

✅ **`withAuth()` ek HOC hai jo authentication logic handle karta hai**
✅ **Reusability increase hoti hai**

4️⃣ Render Props Pattern 🎭

✅ **Jab humein ek component se doosre component ko data ya logic pass karna ho, tab `Render Props` pattern use hota hai**

🔹 **Example (Mouse Position Tracker with Render Props)**

```
function MouseTracker({ render }) {
  const [position, setPosition] = useState({ x: 0, y: 0 });

  useEffect(() => {
    const handleMouseMove = (e) => setPosition({ x: e.clientX, y: e.clientY });
    window.addEventListener("mousemove", handleMouseMove);
    return () => window.removeEventListener("mousemove", handleMouseMove);
  }, []);
```

```
  return render(position);
}

function App() {
  return (
    <MouseTracker render={(pos) => <h2>Mouse Position: {pos.x}, {pos.y}</h2>} />
  );
}
```

✅ **Logic reusable ban jata hai**
✅ **Different components ko same logic reuse karne ki flexibility milti hai**

5️⃣ Error Boundaries in React 🚨

Agar **React component me koi error aa jaye**, to pura app **crash hone ke bajaye** ek **fallback UI show karna** better hota hai.

✅ **Class Components me `componentDidCatch` method se implement hota hai**
✅ **Errors ko properly handle karta hai without breaking the whole app**

🔹 **Example (Error Boundary Component)**

```
class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false };
  }

  static getDerivedStateFromError() {
    return { hasError: true };
  }
```

```
  componentDidCatch(error, errorInfo) {
    console.error("Error Caught:", error, errorInfo);
  }

  render() {
    return this.state.hasError ? <h2>Something went wrong!</h2> : this.props.children;
  }
}

export default ErrorBoundary;
```

🔹 **Using Error Boundary in App**

```
<ErrorBoundary>
  <MyComponent />
</ErrorBoundary>
```

✅ **App crash hone se bach jata hai**
✅ **Error track aur log kiya ja sakta hai**

6️⃣ Lazy Loading & Code Splitting 🎭

Jab **React app ka JavaScript bundle size bohot bada ho jaye**, to **initial page load slow ho sakta hai**. **Lazy Loading & Code Splitting** se **sirf zaroorat ka code load hota hai**.

✅ **Bundle size optimize hota hai**
✅ **Page fast load hota hai**

🔹 **Example (Lazy Loading with `React.lazy()`)**

```
import React, { Suspense } from "react";
```

```
const LazyComponent = React.lazy(() => import("./HeavyComponent"));

function App() {
  return (
    <div>
      <h1>Welcome to My App</h1>
      <Suspense fallback={<h2>Loading...</h2>}>
        <LazyComponent />
      </Suspense>
    </div>
  );
}

export default App;
```

✅ **Lazy loading se `HeavyComponent` tabhi load hoga jab zaroorat hogi**
✅ **Loading state ke liye `Suspense` component ka use kiya gaya hai**

🎯 Conclusion

- **Events handling React ke andar easy aur optimized hai**
- **Debouncing & Throttling performance ko optimize karta hai**
- **HOC aur Render Props code reuse karne ke liye best patterns hain**
- **Error Boundaries React app ko crash hone se bachate hain**
- **Lazy Loading & Code Splitting se app fast aur lightweight banta hai**

🚀 **Advanced React Concepts se apps aur optimized aur scalable banti hain!**

📌 CHAPTER 9: WORKING WITH APIs & DATA FETCHING 🌐

1️⃣ Fetching Data using Fetch API 🔄

React me data fetch karne ke liye `fetch()` **API** ka use hota hai. Ye **built-in JavaScript function** hai jo **asynchronous HTTP requests** bhejta hai aur **promises** return karta hai.

✅ **Lightweight & Simple**
✅ **Built-in JavaScript Function**
✅ **Supports `GET`, `POST`, `PUT`, `DELETE` Requests**

🔹 **Example (Fetching Data using `fetch()`)**

```
import React, { useState, useEffect } from "react";

function FetchData() {
  const [data, setData] = useState([]);

  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/posts")
      .then((response) => response.json())
      .then((data) => setData(data))
      .catch((error) => console.error("Error fetching data:", error));
  }, []);

  return (
    <div>
      <h2>Data from API</h2>
      <ul>
```

```
            {data.map((item) => (
              <li key={item.id}>{item.title}</li>
            ))}
          </ul>
        </div>
    );
}

export default FetchData;
```

✅ **Data API se fetch ho raha hai aur state me store ho raha hai**
✅ `useEffect()` **ensure kar raha hai ke API call sirf ek dafa ho**
✅ **Agar error aaye to console me log ho raha hai**

2️⃣ Fetching Data using Axios 🚀

Axios ek **third-party library** hai jo **fetch API ka alternative** hai. Iska **syntax simple aur error handling better hoti hai**.

✅ **Auto JSON Parsing**
✅ **Built-in Error Handling**
✅ **Supports Request & Response Interceptors**

🔹 **Installation:**

```
npm install axios
```

🔹 **Example (Fetching Data using Axios)**

```
import React, { useState, useEffect } from "react";
import axios from "axios";
```

```
function FetchDataWithAxios() {
  const [data, setData] = useState([]);

  useEffect(() => {
    axios.get("https://jsonplaceholder.typicode.com/posts")
      .then((response) => setData(response.data))
      .catch((error) => console.error("Error fetching data:", error));
  }, []);

  return (
    <div>
      <h2>Data from API</h2>
      <ul>
        {data.map((item) => (
          <li key={item.id}>{item.title}</li>
        ))}
      </ul>
    </div>
  );
}

export default FetchDataWithAxios;
```

✅ **Axios automatically JSON parse kar leta hai**
✅ **Error handling simplified hai**
✅ **Code `fetch()` se cleaner aur readable hai**

3️⃣ Using `useEffect` for Data Fetching 🔄

React me API se data fetch karne ke liye `useEffect` ka use hota hai. Ye **component ke mount hone par API call execute karta hai**.

🔹 **Example (`useEffect` with Fetch API)**

```
useEffect(() => {
  fetch("https://api.example.com/data")
    .then((response) => response.json())
    .then((data) => setData(data))
    .catch((error) => console.error("Error:", error));
}, []);  // Empty dependency array ensures API call runs once
```

✅ **API call sirf ek dafa chalega**
✅ **State update hone ke baad UI render hoga**

4️⃣ Handling API Errors ❌

Jab **API call fail ho jaye**, to **error handling zaroori hoti hai** taake user ko proper feedback mile.

🔹 **Example (Error Handling in Fetch API)**

```
useEffect(() => {
  fetch("https://api.example.com/data")
    .then((response) => {
      if (!response.ok) {
        throw new Error("Network response was not ok");
      }
      return response.json();
    })
    .then((data) => setData(data))
    .catch((error) => setError(error.message));
}, []);
```

🔹 **Example (Error Handling in Axios)**

```
axios.get("https://api.example.com/data")
  .then((response) => setData(response.data))
  .catch((error) => {
    if (error.response) {
      console.error("Server Error:", error.response.data);
    } else if (error.request) {
      console.error("No Response from Server");
    } else {
      console.error("Request Failed:", error.message);
    }
  });
```

✅ **Error messages user ko show karne ke liye state me store kiye ja sakte hain**
✅ **Axios `error.response` kaafi useful hota hai error type detect karne ke liye**

5️⃣ Caching API Data for Performance 🚀

Agar **same API request multiple dafa ho rahi hai**, to **caching se performance improve ho sakti hai**.

🟢 Solution 1: Local State Caching

Agar **data ek dafa fetch ho gaya**, to usse state me store karke **bar-bar API call avoid** ki ja sakti hai.

🔹 **Example (Local Caching in State)**

```
const [data, setData] = useState(() => {
  const cachedData = localStorage.getItem("apiData");
  return cachedData ? JSON.parse(cachedData) : null;
});

useEffect(() => {
```

```
  if (!data) {
    fetch("https://api.example.com/data")
      .then((response) => response.json())
      .then((fetchedData) => {
        setData(fetchedData);
        localStorage.setItem("apiData", JSON.stringify(fetchedData));
      })
      .catch((error) => console.error("Error:", error));
  }
}, []);
```

✅ **Agar data cache me hai to API call avoid hogi**
✅ `localStorage` **data store karta hai jo page refresh pe bhi available rahta hai**

🔵 Solution 2: Caching with React Query

React Query ek **powerful library hai jo automatic API caching** aur **data fetching optimization** karta hai.

🔹 **Installation:**

```
npm install @tanstack/react-query
```

🔹 **Example (Caching with React Query)**

```
import { useQuery } from "@tanstack/react-query";
import axios from "axios";

function FetchDataWithReactQuery() {
  const { data, error, isLoading } = useQuery(["apiData"], () =>
    axios.get("https://api.example.com/data").then((res) => res.data)
  );
```

```
  if (isLoading) return <p>Loading...</p>;
  if (error) return <p>Error fetching data!</p>;

  return (
    <div>
      <h2>Data from API</h2>
      <ul>
        {data.map((item) => (
          <li key={item.id}>{item.title}</li>
        ))}
      </ul>
    </div>
  );
}

export default FetchDataWithReactQuery;
```

✅ **React Query automatic caching aur refetching handle karta hai**
✅ **State management kaafi optimized ho jata hai**

6️⃣ Real-Time Data with WebSockets 📡

Agar **app me real-time updates ki zaroorat ho** (e.g., chat apps, stock prices, live notifications), to **WebSockets ka use hota hai**.

✅ **Server changes ko real-time me UI pe update karta hai**
✅ **Traditional API polling ke mukable faster & efficient hai**

🔹 **Example (Real-Time Data with WebSockets in React)**

```
import React, { useEffect, useState } from "react";
```

```
function WebSocketComponent() {
  const [message, setMessage] = useState("");

  useEffect(() => {
    const socket = new WebSocket("wss://example.com/socket");

    socket.onmessage = (event) => {
      setMessage(event.data);
    };

    return () => socket.close();
  }, []);

  return <h2>Latest Update: {message}</h2>;
}

export default WebSocketComponent;
```

✅ **WebSockets real-time updates ke liye perfect hain**
✅ **Component cleanup me `socket.close()` important hai taake memory leak na ho**

🎯 Conclusion

- **`fetch()` aur `Axios` API fetching ke liye use hote hain**
- **`useEffect()` ensure karta hai ke API call component ke mount hone par ho**
- **Error handling zaroori hai taake user ko proper feedback mile**
- **Data caching se performance optimize hoti hai (`localStorage`, React Query)**
- **Real-time data ke liye WebSockets best solution hai**

🚀 **Efficient API handling React apps ko fast aur user-friendly banata hai!**

📌 CHAPTER 10: REACT PERFORMANCE OPTIMIZATION ⚡

1️⃣ Memoization with `useMemo` 🧠

React me unnecessary re-calculations ko avoid karne ke liye `useMemo` ka use hota hai. Ye ek computed value ko memoize karta hai, jo tabhi re-compute hoti hai jab dependencies change hoti hain.

✅ Expensive computations ko optimize karta hai
✅ Performance improve karta hai

🔹 Example (Expensive Calculation Optimization with `useMemo`)

```
import React, { useState, useMemo } from "react";


function ExpensiveComponent({ number }) {

  const factorial = useMemo(() => {

    console.log("Factorial calculated!");

    let result = 1;

    for (let i = 1; i <= number; i++) {

      result *= i;

    }
```

```
    return result;
  }, [number]); // Ye computation sirf jab number change hoga tab hoga


  return <h2>Factorial of {number} is {factorial}</h2>;
}


export default ExpensiveComponent;
```

✅ Factorial computation sirf jab `number` change hoga tab execute hoga
✅ `useMemo` unnecessary calculations avoid karta hai

2️⃣ Avoiding Unnecessary Re-renders 🔄

Agar React component unnecessary re-render ho raha hai, to performance slow ho sakti hai.

🔹 Common Reasons for Unnecessary Re-renders:

❌ State/Props ka bina zaroorat update hona
❌ Parent component ke re-render hone se child component ka bhi re-render hona
❌ Functions aur objects ka har render pe naye reference create hona

✅ Solution: `React.memo()` ka use karein

`React.memo()` se component sirf tab re-render hoga jab uski props change hongi.

🔹 Example (Optimizing Re-renders with `React.memo`)

```
import React from "react";


const MyComponent = React.memo(({ name }) => {
  console.log("Component rendered!");
  return <h2>Hello, {name}</h2>;
});


export default MyComponent;
```

✅ Agar `name` prop same hai, to re-render nahi hoga
✅ Performance boost hoti hai

3️⃣ Using `useCallback` to Optimize Functions 🚚

Jab functions unnecessary re-create hote hain, to child components re-render hote hain. Is problem ko `useCallback` solve karta hai.

✅ Function ka reference store karta hai taake har render pe naye function create na ho
✅ Performance improve karta hai jab function child component ko pass hota hai

🔹 Example (Optimizing Functions with `useCallback`)

```
import React, { useState, useCallback } from "react";


function Parent() {
```

```
  const [count, setCount] = useState(0);


  const increment = useCallback(() => {
    setCount((prev) => prev + 1);
  }, []); // Ye function har render pe recreate nahi hoga


  return (
    <div>
      <h2>Count: {count}</h2>
      <Child increment={increment} />
    </div>
  );
}


const Child = React.memo(({ increment }) => {
  console.log("Child component re-rendered!");
  return <button onClick={increment}>Increment</button>;
});


export default Parent;
```

✅ `increment` function har render pe naye reference se create nahi hoga
✅ Child component unnecessary re-render se bach jayega

4️⃣ React Profiler Tool 📊

React me `Profiler` performance bottlenecks detect karne ka tool hai. Ye components ke render time ko track karta hai, aur optimization suggestions deta hai.

🔹 Profiler Setup in React Developer Tools

1️⃣ React Developer Tools Chrome Extension Install karein
2️⃣ Profiler tab me `Record` button click karein
3️⃣ Interaction perform karein aur re-renders analyze karein

✅ Profiler performance debugging ke liye best tool hai
✅ Render time aur unnecessary renders ko detect karta hai

5️⃣ Code Splitting with `React.lazy()` 🎭

Jab React app ka JavaScript bundle size bohot bada ho jaye, to initial page load slow ho sakta hai. Iska solution hai Code Splitting!

✅ Lazy loading se sirf required components load hote hain
✅ Bundle size optimize hota hai
✅ Page load fast hota hai

🔹 Example (Code Splitting with `React.lazy()`)

```
import React, { Suspense } from "react";
```

```
const LazyComponent = React.lazy(() => import("./HeavyComponent"));


function App() {
  return (
    <div>
      <h1>Welcome to My App</h1>
      <Suspense fallback={<h2>Loading...</h2>}>
        <LazyComponent />
      </Suspense>
    </div>
  );
}


export default App;
```

✅ `LazyComponent` tabhi load hoga jab zaroorat hogi
✅ Bundle size optimize ho jayega
✅ Loading state dikhane ke liye `Suspense` ka use kiya gaya hai

🎯 Conclusion

- `useMemo` se expensive calculations optimize hoti hain
- `React.memo()` unnecessary re-renders ko avoid karta hai

- `useCallback()` **functions ke unnecessary recreation ko stop karta hai**
- **React Profiler se performance bottlenecks detect kiye jate hain**
- **Code Splitting (`React.lazy()`) se bundle size optimize hota hai**

🚀 **Performance optimization React apps ko fast aur efficient banata hai!**

📌 CHAPTER 11: AUTHENTICATION & SECURITY IN REACT 🔐

1️⃣ Handling Authentication in React 🚦

Authentication ka matlab hai **user ki identity verify karna**. React applications me authentication **3 common methods** se hoti hai:

✅ **Session-Based Authentication** – Server pe session store hoti hai
✅ **Token-Based Authentication (JWT)** – Server ek token generate karta hai jo client store karta hai
✅ **Third-Party Authentication** – Google, Facebook, GitHub login etc.

🟢 Basic Authentication Flow in React

1️⃣ **User login karega (email/password enter karega)**
2️⃣ **Backend se credentials verify hoke token generate hoga**
3️⃣ **Frontend me token store hoga (LocalStorage/SessionStorage)**
4️⃣ **Every API request ke saath token send hoga for authentication**

2️⃣ Using JWT for Authentication 🔑

JWT (JSON Web Token) ek **token-based authentication method** hai jo **secure aur stateless hota hai**.

✅ **User authenticate hone ke baad server ek JWT issue karta hai**
✅ **JWT ek base64 encoded string hoti hai jo 3 parts me divided hoti hai:**

Part	Description
Header	Token ka type aur hashing algorithm
Payload	User ki information (ID, email, role, etc.)
Signature	Token ko verify karne ke liye hash

🔹 **JWT Structure Example:**

```
{
 "header": {
  "alg": "HS256",
  "typ": "JWT"
 },
 "payload": {
  "user_id": "123",
  "email": "user@example.com",
  "role": "admin"
 },
 "signature": "hashed_signature"
}
```

🟢 Implementing JWT Authentication in React

1️⃣ User Login Form Create Karna

```
import { useState } from "react";
```

```
function Login() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");

  const handleLogin = async () => {
    const response = await fetch("https://api.example.com/login", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ email, password }),
    });

    const data = await response.json();
    if (data.token) {
      localStorage.setItem("authToken", data.token);
      alert("Login Successful!");
    }
  };

  return (
    <div>
      <input type="email" placeholder="Email" onChange={(e) =>
setEmail(e.target.value)} />
      <input type="password" placeholder="Password" onChange={(e) =>
setPassword(e.target.value)} />
      <button onClick={handleLogin}>Login</button>
    </div>
  );
}

export default Login;
```

✅ **User credentials API ko bheje ja rahe hain**
✅ **Agar response me token aaye to `localStorage` me save ho raha hai**

3️⃣ Protecting Routes in React 🚫

JWT authentication ke baad **protected routes create karna zaroori hota hai** taake unauthorized users pages access na kar sakein.

🔹 **Example (Protected Route Component)**

```
import { Navigate } from "react-router-dom";

function ProtectedRoute({ children }) {
  const token = localStorage.getItem("authToken");
  return token ? children : <Navigate to="/login" />;
}

export default ProtectedRoute;
```

🔹 **Protected Routes Apply Karna**

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
import Dashboard from "./Dashboard";
import Login from "./Login";
import ProtectedRoute from "./ProtectedRoute";

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/login" element={<Login />} />
        <Route path="/dashboard" element={
```

```
            <ProtectedRoute>
              <Dashboard />
            </ProtectedRoute>
          } />
        </Routes>
      </BrowserRouter>
    );
}

export default App;
```

✅ **Agar user authenticated hai to hi Dashboard dikhaya jayega**
✅ **Agar token nahi mila to user ko `/login` par redirect kar diya jayega**

4️⃣ Storing Tokens Securely 🔒

Token store karne ke **2 common methods** hain:

Storage	Security
LocalStorage	Less secure, XSS attacks ke liye vulnerable
HttpOnly Cookies	More secure, Server-side stored, CSRF safe

🟢 LocalStorage Usage (Less Secure, Avoid for Sensitive Data)

```
localStorage.setItem("authToken", token);  // Store token
const token = localStorage.getItem("authToken");  // Get token
localStorage.removeItem("authToken");  // Remove token
```

⚠️ **LocalStorage me token store karna XSS attacks ke liye vulnerable ho sakta hai**

🔵 HttpOnly Cookies Usage (More Secure)

HttpOnly cookies **backend set karta hai**, aur **JavaScript access nahi kar sakti** (XSS attacks se safe).

🔹 **Backend Response Example:**

```
res.cookie("authToken", token, {
  httpOnly: true, // JavaScript se access nahi ho sakta
  secure: true, // HTTPS par hi chalega
  sameSite: "Strict", // CSRF attack prevent karega
});
```

✅ **Token secure server ke through manage hota hai**
✅ **CSRF aur XSS attacks se better protection milti hai**

5️⃣ Role-Based Access Control (RBAC) 🏆

Har user ka **role different hota hai** (e.g. Admin, User, Editor). RBAC ka use kar ke **restricted areas ko secure kiya jata hai**.

🔹 **Example (RBAC Implementation in React)**

```
function ProtectedRoute({ children, role }) {
  const token = localStorage.getItem("authToken");
  const user = token ? JSON.parse(atob(token.split(".")[1])) : null;

  if (!user) return <Navigate to="/login" />;
  if (role && user.role !== role) return <h1>Access Denied ❌</h1>;
```

```
  return children;
}
```

🔹 **Applying Role-Based Access**

```
<Routes>
  <Route path="/admin" element={
    <ProtectedRoute role="admin">
      <AdminDashboard />
    </ProtectedRoute>
  } />
</Routes>
```

✅ **Agar user ka role "admin" nahi hai to "Access Denied" show hoga**
✅ **RBAC se unauthorized users ko restricted pages se block kiya jata hai**

🎯 Conclusion

- **JWT authentication React apps me secure login system provide karta hai**
- **Protected Routes unauthorized users ko restricted pages se block karte hain**
- **Tokens ko secure rakhna zaroori hai (HttpOnly Cookies > LocalStorage)**
- **RBAC ka use karke different user roles ko manage kiya jata hai**

🚀 **Secure authentication se React apps safe aur efficient banti hain!**

📌 CHAPTER 12: DEPLOYING REACT APPLICATIONS 🚀

1️⃣ Building a Production-Ready React App 🏗️

Jab aapka React project **development phase** se complete ho jaye, to usko **production ke liye optimize** karna zaroori hota hai. Production me **performance aur security best practices** follow karni chahiye.

✅ **Code ko optimize karna** (Tree shaking, minification, etc.)
✅ **Assets aur images ko compress karna**
✅ **Environment variables set karna**

🔹 **React App Ko Build Karna (Production Ready Code Generate Karna)**

npm run build

Is command ka output:

- `build/` folder create hota hai jisme **optimized aur minified JavaScript aur CSS files hoti hain**
- Ye folder **production ke liye ready hota hai**

2️⃣ Hosting React Apps on Netlify, Vercel, and GitHub Pages 🌍

React applications ko **deploy karne ke liye different free hosting platforms** available hain.

Hosting Service	Features
Netlify	Fast deployment, automatic CI/CD, custom domains
Vercel	Serverless functions support, instant rollback
GitHub Pages	Free hosting for static sites, works with GitHub repos

---

🟢 Deploying on Netlify

Netlify ek **fast aur easy** hosting platform hai jo automatic deployments support karta hai.

🔹 **Deploy React App on Netlify (Using CLI)**

1️⃣ **Netlify CLI install karein:**

```
npm install -g netlify-cli
```

2️⃣ **Build folder generate karein:**

```
npm run build
```

3️⃣ **Netlify par deploy karein:**

```
netlify deploy --prod
```

4️⃣ **Netlify ka interactive setup follow karein aur deployment complete ho jayega.**

✅ **Live URL mil jata hai jisme aapki React app accessible hoti hai**
✅ **Custom domains aur SSL certificate free milta hai**

🔵 Deploying on Vercel

Vercel ek **performance-focused hosting service hai**, jo **Next.js aur React** applications ke liye best hai.

🔹 **Vercel par React App Deploy Karna**

1️⃣ **Vercel CLI install karein:**

```
npm install -g vercel
```

2️⃣ **Build folder generate karein:**

```
npm run build
```

3️⃣ **Vercel par deploy karein:**

```
vercel
```

4️⃣ **Vercel ka setup follow karein aur automatic deployment complete ho jayega.**

✅ **Automatic HTTPS aur domain setup**
✅ **CI/CD support aur rollback option**

🔴 Deploying on GitHub Pages

GitHub Pages **static websites host karne ke liye use hota hai**. Agar aapki React app **client-side rendering pe depend karti hai**, to **GitHub Pages pe deploy karna best option ho sakta hai**.

🔹 **GitHub Pages ke through Deploy Karna**

1️⃣ **React app ke `package.json` file me `homepage` add karein:**

```
"homepage": "https://username.github.io/repository-name"
```

2️⃣ **GitHub Pages package install karein:**

```
npm install gh-pages --save-dev
```

3️⃣ **`package.json` ke `scripts` section me ye lines add karein:**

```
"predeploy": "npm run build",
"deploy": "gh-pages -d build"
```

4️⃣ **App ko deploy karein:**

```
npm run deploy
```

✅ **GitHub Pages par free hosting milti hai**
✅ **Static React apps ke liye best option**

3️⃣ Setting Up Continuous Deployment (CD) 🔄

Continuous Deployment ka matlab hai **jab bhi aap code update karein, automatically naye changes deploy ho jayein**. Ye process **CI/CD pipelines** ke zariye hoti hai.

✅ **Every push ke saath automatic deployment hota hai**
✅ **Manually deploy karne ki zaroorat nahi hoti**

🟢 Setting Up CD with GitHub & Netlify

1️⃣ **GitHub par apni repo ko push karein**
2️⃣ **Netlify pe login karein aur "New Site from Git" option select karein**
3️⃣ **GitHub repo connect karein aur branch select karein**
4️⃣ **Netlify automatically har naye commit par changes deploy karega**

✅ **Zero manual deployment required**
✅ **Auto rollback ka option available hai**

🔵 Setting Up CD with GitHub & Vercel

1️⃣ **GitHub repository connect karein with Vercel**
2️⃣ **Vercel automatically branch changes detect karega**
3️⃣ **Every commit ke saath latest build deploy ho jayega**

✅ **Vercel instant deployment deta hai**
✅ **Multiple environments ka support available hai**

4️⃣ Performance Optimization Before Deployment ⚡

Agar app ko **deploy karne se pehle optimize** na kiya jaye, to **slow loading aur higher bandwidth usage** ho sakta hai.

✅ Essential Optimization Steps:

1️⃣ Code Minification & Tree Shaking

- React **production mode me unnecessary code remove** karta hai

JS aur CSS ko minify karne ke liye:

```
npm run build
```

-

2️⃣ Image Optimization

- **JPG/PNG images ko compress karein aur WebP format use karein**

React Lazy Loading ka use karein:

```
import myImage from "./image.webp";
<img src={myImage} loading="lazy" alt="Optimized Image" />;
```

-

3️⃣ Lazy Loading & Code Splitting

- React me `React.lazy()` aur `Suspense` ka use karein

Lazy Loading Example:

```
const Home = React.lazy(() => import("./Home"));
```

-

4️⃣ Reduce Unused Dependencies

Junk dependencies remove karne ke liye:

```
npm prune
```

-

5️⃣ Caching & CDN Use Karein

- **CDN se assets ko faster load kiya ja sakta hai**

- **Netlify aur Vercel automatically CDN caching karte hain**

🎯 Conclusion

- **React apps deploy karne ke liye Netlify, Vercel aur GitHub Pages best options hain**
- `npm run build` **se optimized production build generate hota hai**
- **CD/CD (Continuous Deployment) se automatic deployment possible hoti hai**
- **Deployment se pehle performance optimize karna zaroori hai**

🚀 **Properly deployed React app fast aur scalable hoti hai!**